Fynla Mobile App Exploration

Date: 6 March 2026 **Status:** Design exploration (pre-implementation) **Current Version:** v0.8.3 (web)

Executive Summary

This document explores building a Fynla mobile app from four perspectives: UX design, technical architecture, security/data protection, and a devil's advocate challenge. The analysis is grounded in the existing codebase (378 Vue components, 1,037-line API routes file, 70 controllers, 7 financial modules) and the fynlaDesignGuide.md v1.2.0 design system.

Recommendation: A phased approach starting with a **Progressive Web App (PWA)** that evolves into a **Capacitor-wrapped hybrid app** if mobile adoption justifies it. This maximises code reuse from the existing Vue.js 3 codebase while providing native device access (biometrics, push notifications, camera) when needed.

---

1. UX Analysis

1.1 Navigation Architecture

The current web app uses a collapsible sidebar (`SideMenu.vue`) with sections: Main (Dashboard, Net Worth), Current (Cash, Protection, Investments, Retirement), Planning (Savings, Goals, Estate, Trusts), and Analysis (Risk, UK Taxes, Holistic Plan). This maps well to mobile:

Recommended: Bottom Tab Bar + Hub-and-Spoke

Tab	Content	Icon
Dashboard	Overview, net worth summary	Home
Modules	Hub linking to 7 modules	Grid
Goals	Goals & life events (high engagement)	Target
Actions	Recommendations, alerts, tasks	Bell
Profile	Settings, account, help	User

Each module opens as a full-screen stack from the Modules hub. This avoids cramming 12+ sidebar items into a bottom bar while keeping the most-used features one tap away.

1.2 Touch-First Interactions

- **Touch targets:** Minimum 44x44pt (Apple HIG) / 48x48dp (Material). Current web buttons (`px-4 py-2`) need enlargement for mobile
- **Swipe gestures:** Swipe-to-delete on policies/accounts, swipe between tabs in detail views
- **Pull-to-refresh:** On all data-heavy views (dashboard, account lists, goal progress)
- **Long-press:** Quick actions on list items (edit, share, archive)

- **Haptic feedback:** On successful form submissions, goal milestone achievements, payment confirmations

1.3 Data Density on Small Screens

Financial planning involves complex tables, multi-column data, and dense forms. Mobile strategy:

- **Summary cards with drill-down:** Dashboard shows top-level metrics (total net worth, protection gap, pension progress). Tap to expand into full detail
- **Progressive disclosure:** Show the 3 most important fields first, "Show more" for the rest
- **Horizontal scroll for tables:** Use horizontally-scrollable cards for comparison tables (e.g., pension comparison, ISA rates)
- **Stacked layout:** Convert side-by-side layouts to vertical stacks (the design guide already specifies `grid-cols-1 md:grid-cols-2 lg:grid-cols-3`)
- **Charts:** ApexCharts (currently used) is responsive. Enable touch zoom/pan. Simplify legends on mobile to bottom-positioned, scrollable

1.4 Mobile-Native Features (Leveraging Device Capabilities)

Feature	Purpose	Priority
Biometric login (Face ID / Touch ID / Fingerprint)	Replace email verification code for returning users	High
Push notifications	Goal milestones, policy renewals, contribution reminders, market alerts	High
Camera / Document scanning	Scan pension statements, policy documents, payslips for data extraction	Medium
Haptic feedback	Confirmation of saves, milestone celebrations	Medium
Widgets (iOS/Android)	Quick-glance net worth, goal progress on home screen	Medium
Shortcuts / Quick Actions	3D Touch / long-press app icon: "Add Transaction", "View Goals"	Low
NFC	Future: tap-to-share financial summary with adviser	Low
Siri / Google Assistant	"Hey Siri, what's my net worth?"	Low

1.5 Forms on Mobile

Current forms (property details, pension info, policy details) have 10-20+ fields. Mobile approach:

- **Multi-step wizard pattern** (already exists in onboarding): Break long forms into 3-5 step wizards with progress indicator
- **Smart defaults:** Pre-fill UK-specific defaults (e.g., tax year, ISA allowance)
- **Keyboard optimisation:** Numeric keyboard for currency inputs, email keyboard for email fields
- **Inline validation:** Validate each field on blur, show errors immediately (not on submit)
- **Auto-save drafts:** Save form progress locally so users can resume later

1.6 Fyn Mascot on Mobile

The springbok character "Fyn" can play a more prominent role on mobile:

- **Onboarding guide:** Fyn walks users through initial setup
- **Achievement celebrations:** Animated Fyn for goal milestones
- **Empty states:** Fyn with contextual message when no data exists
- **Loading states:** Subtle Fyn animation instead of generic spinner (for longer loads)
- Keep Fyn small and non-intrusive - a small avatar in guidance tooltips, not a full-screen character

1.7 Offline Experience

- **Read-only dashboard:** Cache last-known net worth, goal progress, policy summaries
- **Queued actions:** Allow users to add transactions/notes offline, sync when connected
- **Clear offline indicator:** Subtle banner when offline, with "Last updated" timestamp
- **Critical data available:** Tax reference data, calculator tools should work offline

---

2. Technical Architecture

2.1 Framework Comparison

Criteria	PWA (Vue.js)	Capacitor + Vue	React Native	Flutter	Native (Swift/Kotlin)
Code reuse with web	100%	90-95%	10-20%	0%	0%
Native device access	Limited	Full (via plugins)	Full	Full	Full
App store distribution	No (web only)	Yes	Yes	Yes	Yes
Learning curve	None	Low	Medium	High	High
Performance	Good	Good	Good	Excellent	Excellent
Team skill match	Perfect (Vue.js)	Perfect (Vue.js)	JS but not Vue	Dart (new)	Swift/Kotlin (new)
Maintenance burden	Minimal	Low-medium	High	High	Very high (2 codebases)
Estimated effort	2-4 weeks	6-10 weeks	16-24 weeks	16-24 weeks	24-40 weeks

Recommended: Capacitor + Vue.js 3 (Phase 2, after PWA proves demand)

Capacitor wraps the existing Vue.js app in a native shell, providing access to device APIs (biometrics, camera, push notifications, haptics) while reusing 90-95% of existing code. The web app continues to work as-is.

2.2 API Strategy

The existing Laravel API (1,037 lines in `routes/api.php`, 70 controllers) is already well-structured for mobile consumption:

- **RESTful endpoints** with Sanctum token auth
- **Resource classes** for consistent JSON responses
- **Form request validation** (70 classes) handles input validation server-side

Recommendations:

- **Keep the existing API.** No BFF layer needed initially - the API is already JSON-first
- **Add API versioning:** Prefix routes with `/api/v1/` to allow future breaking changes without affecting mobile users on older app versions
- **Add response compression:** Enable gzip/brotli for API responses (reduces payload 60-80%)
- **Add ETags / conditional requests:** For cached data (tax configs, market rates) to reduce bandwidth
- **Add pagination metadata:** Ensure all list endpoints return `{ data: [], meta: { current_page, total, per_page } }` for infinite scroll on mobile

2.3 Authentication Flow for Mobile

Current flow: email + password -> verification code -> session token.

Mobile-enhanced flow:

1. First login: email + password -> verification code -> Sanctum token stored in secure storage
2. Subsequent logins: biometric unlock -> stored token refresh
3. Token refresh: automatic refresh before expiry (Sanctum tokens with 30-day expiry, refresh at 25 days)
4. Session timeout: after 15 minutes of inactivity, require biometric re-auth (not full login)
5. Device registration: track device IDs for multi-device management

2.4 Offline Data Architecture

```
Mobile App
  |-- Local Cache (SQLite via Capacitor)
  |     |-- Dashboard summary (cached on each sync)
  |     |-- Goal progress snapshots
  |     |-- Policy/account list metadata
  |     |-- Tax reference data (yearly update)
  |     |-- Pending offline actions queue
  |
  |-- Sync Manager
  |     |-- Background sync when connectivity restored
  |     |-- Conflict resolution (server wins for calculations)
  |     |-- Delta sync (only changed records since last sync timestamp)
  |
  |-- API Layer (existing services/)
        |-- Axios interceptor adds offline queue support
        |-- Retry logic already exists (exponential backoff)
```

2.5 State Management

The existing 21 Vuex modules can be reused directly with Capacitor. Add:

- **Persistence plugin:** Vuex state persisted to Capacitor Storage (encrypted)
- **Sync status tracking:** Each module tracks `lastSynced` timestamp
- **Optimistic updates:** Apply changes locally, sync to server, rollback on failure

2.6 Build & Deploy Pipeline

Stage	Tool	Notes
Local build	Existing Vite config	Add Capacitor build target
iOS build	Xcode (via Capacitor CLI)	Requires macOS, Apple Developer account (GBP 79/year)
Android build	Android Studio (via Capacitor CLI)	Free
CI/CD	GitHub Actions	Automate builds on tagged releases
iOS distribution	TestFlight -> App Store	Review process: 1-7 days for financial apps
Android distribution	Google Play Console	Review process: 1-3 days
OTA updates	Capacitor Live Update (Appflow) or Capgo	Push web layer updates without store review

2.7 Project Structure (Capacitor)

```
fynla/
  |-- resources/js/          (existing Vue.js app - shared)
  |-- mobile/
  |     |-- ios/             (Xcode project, auto-generated)
  |     |-- android/         (Android Studio project, auto-generated)
  |     |-- capacitor.config.ts
  |     |-- plugins/
  |           |-- biometrics.ts
  |           |-- push-notifications.ts
  |           |-- camera.ts
  |           |-- haptics.ts
  |-- resources/js/utils/
        |-- platform.js      (detect web vs mobile, expose device APIs)
```

---

3. Security & Data Protection

3.1 Regulatory Landscape

Regulation	Requirement	Impact on Mobile
UK GDPR / DPA 2018	Lawful basis for processing, data minimisation, right to erasure	Must handle data deletion when app uninstalled or account deleted
FCA (if applicable)	Financial promotions rules, clear risk warnings	App store screenshots and descriptions must comply
ICO Mobile App Guidance	Privacy by design, transparent data collection	Privacy policy accessible before first data entry
PCI DSS	Card data protection (Revolut handles this via their SDK)	Never store card details locally. Revolut SDK handles PCI scope
Apple App Store Guidelines	4.2 (financial app requirements), 5.1 (privacy)	Must declare all data collection in App Store privacy nutrition labels
Google Play Policies	Financial services policy, data safety section	Must complete data safety form accurately

3.2 Authentication Security

Current state: Laravel Sanctum with email verification codes, session-based auth.

Mobile additions required:

Control	Implementation	Priority
Biometric auth	iOS: Face ID / Touch ID via LocalAuthentication. Android: BiometricPrompt API. Used for returning-user unlock, NOT as sole authentication	Critical
Secure token storage	iOS: Keychain with `kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly`. Android: EncryptedSharedPreferences backed by Android Keystore	Critical
Certificate pinning	Pin the API domain's public key (not cert) to prevent MITM. Use Capacitor HTTP plugin with pinning support	Critical
Session timeout	15-minute inactivity timeout triggers biometric re-auth. Full re-login after 30 days	High
Device binding	Register device ID with server. Alert user on new device login. Allow remote device revocation	High
Step-up auth	Require verification code (not just biometric) for: changing email/password, adding payment methods, exporting data, deleting account	High
Jailbreak/root detection	Warn users on compromised devices. Don't block entirely (false positives) but log and restrict sensitive operations	Medium

3.3 Data Protection on Device

What to store locally:

- Authentication tokens (encrypted in Keychain/Keystore)
- Cached dashboard summary (encrypted)
- Offline action queue (encrypted)
- User preferences (non-sensitive)

What to NEVER store locally:

- Raw financial data (full account details, transaction history)
- Tax calculations or projections
- Personal documents
- Payment credentials
- Passwords or verification codes

Device-level protections:

Protection	iOS	Android
Encrypted storage	Keychain Services	EncryptedSharedPreferences
Screenshot prevention	`UIScreen.main.isCaptured` detection + overlay	`FLAG_SECURE` on sensitive Activities
Clipboard timeout	Clear clipboard after 30 seconds if financial data copied	Same
Backup exclusion	Set `isExcludedFromBackup = true` on sensitive files	Set `android:allowBackup="false"` or exclude specific files
App backgrounding	Show blur overlay when app moves to background	Same
Idle data purge	Clear cached data after 7 days without use	Same

3.4 Network Security

- **TLS 1.3 minimum** with App Transport Security (iOS) / Network Security Config (Android)
- **Certificate pinning** on all API requests (pin public key hash, not certificate, to survive cert rotation)
- **Request signing:** HMAC signature on sensitive write operations (transfers, payment actions)
- **No sensitive data in URLs:** All financial data in POST bodies, never query strings (prevents logging leakage)
- **API rate limiting:** Already exists in Laravel, ensure mobile token-based limits are appropriate

3.5 Audit & Monitoring

- **Device-aware audit logs:** Extend existing `Auditable` trait to include device type, OS version, app version
- **Anomaly detection:** Flag unusual patterns: new device, unusual location, bulk data access, rapid account switching

- **Crash reporting:** Use a privacy-respecting service (e.g., Sentry with PII scrubbing). Never include financial data in crash reports
- **Remote wipe:** Server-side flag that forces token revocation and local data deletion on next app open

3.6 GDPR Right to Erasure on Mobile

When a user deletes their account (existing `GDPRController`):

1. Server deletes all user data (existing)
2. Push notification to all registered devices: trigger local data wipe
3. Revoke all device tokens
4. Clear Keychain/Keystore entries
5. App shows "Account deleted" screen on next open

---

4. Devil's Advocate

4.1 Is a Mobile App Even Needed?

Challenge: Financial planning is a sit-down, considered activity. Users don't plan their pension on the bus. The primary use case is: sit at a computer, open spreadsheets/statements, enter data carefully. Mobile is for checking, not planning.

Counter-argument: True for data entry, but mobile excels at:

- Quick dashboard checks ("What's my net worth today?")
- Goal progress monitoring
- Receiving and acting on notifications (policy renewal reminders)
- Document scanning (camera) to capture statements on the go

Verdict: A mobile app adds value for *monitoring and notifications*, not for *primary data entry*. This means the mobile experience should be read-heavy, not write-heavy.

4.2 The v0.8.3 Problem

Challenge: Fynla is pre-1.0. Building a mobile app before the web app is mature means:

- Two platforms to fix bugs on during the stabilisation phase
- Feature requests on mobile that the web doesn't have yet
- User expectations of feature parity that can't be met
- Developer focus split during the most critical phase

Recommendation: Reach v1.0 on web first. The PWA approach lets you test mobile demand without committing to a full mobile codebase.

4.3 Cost-Benefit Reality

Item	Estimated Cost
Apple Developer Program	GBP 79/year

Item	Estimated Cost
Google Play Developer	GBP 20 (one-time)
Capacitor setup + native plugins	6-10 weeks development
App store review compliance	2-4 weeks (financial apps face extra scrutiny)
Ongoing maintenance (2 platforms)	20-30% additional effort per feature
Push notification infrastructure	GBP 0-50/month (Firebase free tier likely sufficient)

Break-even analysis: At GBP 10.99/month (Standard plan), you need ~15-20 mobile-only subscribers to justify the annual maintenance cost. But most mobile users will be *existing* web users accessing from a second device, not *new* users attracted by the app store presence.

4.4 The PWA Alternative (Recommended Phase 1)

A PWA delivers 70-80% of mobile app value at 10% of the cost:

Feature	PWA	Native/Capacitor
Home screen icon	Yes	Yes
Offline caching	Yes (Service Worker)	Yes
Push notifications	Yes (Web Push, iOS 16.4+)	Yes
Biometric login	No (Web authn limited)	Yes
Camera access	Yes (getUserMedia)	Yes (better quality)
App store presence	No	Yes
Install friction	Very low (banner)	Higher (store download)
Update friction	Zero (automatic)	Low-medium (store updates)
Haptic feedback	No	Yes
Widgets	No	Yes

Missing from PWA: Biometrics, haptics, widgets, app store discoverability. These are nice-to-haves, not deal-breakers for a financial planning tool.

4.5 App Store Risks for Financial Apps

- **Apple review:** Financial apps face enhanced review. Expect 1-2 rejections before approval. Common rejection reasons: missing disclaimers, unclear data handling, financial advice without proper licensing
- **Apple's 30% cut:** If subscriptions are purchased in-app, Apple takes 30% (year 1) / 15% (year 2+). At GBP 10.99/month, that's GBP 3.30/month to Apple. Fynla currently uses Revolut for payments - you'd need to either use Apple IAP (losing margin) or use "reader app" exemption (complex)
- **Rating pressure:** A 3-star financial app loses trust instantly. Need to launch polished or not at all

- **Ongoing compliance:** Both stores update financial app requirements periodically. Must monitor and adapt

4.6 When WOULD a Mobile App Make Sense?

A native/Capacitor app is justified when:

1. **Web app reaches v1.0+** with stable features and low bug rate
2. **Analytics show >30% of web traffic from mobile devices** (measure this first!)
3. **User research confirms mobile demand** (survey existing users)
4. **Biometric login is a top user request** (the #1 feature PWA can't deliver well)
5. **Push notification engagement** from PWA exceeds 20% (proves mobile notification value)
6. **Revenue supports it:** At least 500+ paying subscribers to justify the ongoing cost

---

5. Recommended Phased Approach

Phase 1: PWA (Weeks 1-4) - LOW RISK

- Add Service Worker for offline caching (dashboard, read-only data)
- Add Web App Manifest for home screen installation
- Implement Web Push notifications (goal reminders, policy renewals)
- Optimise existing responsive design for mobile (touch targets, form wizards)
- Add camera access via getUserMedia for document scanning
- **Measure:** Install rate, mobile usage patterns, notification engagement

Phase 2: Capacitor Hybrid (Weeks 5-14) - MEDIUM RISK

Only if Phase 1 metrics justify it:

- Wrap Vue.js app with Capacitor
- Add biometric authentication (Face ID / Touch ID)
- Add native push notifications (FCM/APNs)
- Add haptic feedback
- Submit to App Store and Google Play
- Implement secure local storage (Keychain/Keystore)
- **Measure:** App store downloads, retention, biometric usage, review ratings

Phase 3: Enhanced Native Features (Weeks 15-24) - OPTIONAL

Only if Phase 2 shows strong adoption:

- Home screen widgets (net worth, goal progress)
- Quick Actions (3D Touch / long-press)
- Siri/Google Assistant integration
- Apple Watch / Wear OS companion (read-only dashboard)
- Advanced offline mode with background sync

---

6. Design System Application to Mobile

All mobile UI must follow `fynlaDesignGuide.md` v1.2.0:

Colour Palette (unchanged)

- CTAs/buttons: `raspberry-500` (#E83E6D)
- Text/navigation: `horizon-500` (#1F2A44)
- Success/growth: `spring-500` (#20B486)
- Warnings/focus: `violet-500` (#5854E6)
- Hover/subtle: `savannah-100` (#FDFAF7)
- Backgrounds: `eggshell-500` (#F7F6F4)
- Banned: amber, orange, mustard, neons, pure black

Typography

- Primary: Segoe UI (system font on Windows, fallback on iOS/Android to SF Pro / Roboto)
- Mobile adjustment: Consider using SF Pro (iOS) and Roboto (Android) as primary for native feel, with Segoe UI for brand consistency in marketing materials
- Minimum body text: 16px (prevents iOS auto-zoom on inputs)

Mobile-Specific Patterns

- Bottom tab bar: `bg-white border-t border-light-gray`, active tab `text-raspberry-500`, inactive `text-neutral-500`
- Pull-to-refresh indicator: `raspberry-500` spinner
- Toast notifications: Existing design system pattern, position at top on mobile (avoid bottom bar overlap)
- FABs: Already defined in design guide - use `raspberry-500` FAB for primary actions on list views

---

7. Key Decisions Needed

1. **Measure mobile traffic first?** Add analytics to understand current mobile web usage before investing
2. **PWA first or jump to Capacitor?** Recommendation: PWA first (lower risk, proves demand)
3. **Biometric priority:** Is replacing email verification codes the top user pain point?
4. **Payment strategy:** Use Apple/Google IAP (lose 15-30% margin) or external payments (complex compliance)?
5. **v1.0 gate:** Should the web app reach 1.0 before any mobile work begins?
6. **Document scanning:** Build OCR for pension statements, or defer to manual entry?